Python 02: Working with files and the OS

https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM12.html

Quick links:
Click on any ⌂ to get back here!

- Opening a file
- Reading file contents
- rstrip()
- split()
- Writing to files
- Closing files
- with
- "quick summary"
- About paths
- Basic file manipulation using `os`
- dot tab tab
- Making a folder/directory
- Copying and trees with `shutil`
- Moving and renaming
- Deleting stuff
- Doing a system call
- subprocess module to keep the outputs
- Getting user input
- Getting user input from the Linux command line

Exercises

1. "Mission One": Make some fasta sequences
2. "Mission Two": Running BLAST

- important git/GitHub stuff at the end

Files are important in bioinformatics. We have many file formats:

FASTA

```
>gi|3255998|emb|AJ223353.1| Homo sapiens mRNA for histone H2B, clone pJG4-5-15
GGGGAGTGTGCTAACTATTAACGCTACGATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGC
TCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGGAAGAAGCGCAAGCGCAGCCGCAAGGAGAGCT
ATTCAGTGTATGTGTACAAGGTGCTGAAGCAGGTCCATCCCGACACCGGCATCTCTTCCAAGGCAATGGG
GATCATGAATTCCTTCGTCAACGACATCTTCGAGCGCATCGCAGGCGAGGCTTCCCGCCTGGCGCATTAC
AACAAGCGCTCGACCATCACCTCCAGGGAGATCCAGACGGCCGTGCGCCTGCTGCTTCCGGGGGAGCTGG
CCAAGCACGCCGTGTCGGAGGGCACCAAGGCCGTCACCAAGTACACCAGTTCCAAGTAACTTTGCCAAGG
GAGAGACATGAAGACAGAGGAGAAATGAATGCATAAAATAACTGATAATATGAATCTATACATAGAACTT
AGGAAGTCTCATCTGCCTGAAAATGACTGTGTGGATCCCACCCAAATCCAACTCATCCTGGTTTGCTGCA
CACTGGTTCATCAAAAGAAGGTTACCGAGGGGAAGGAACTAAAGGTGTTTGCACTTCATGTTACTTTTTG
AGTTTATAAACATAAAAACAGAATTTACTTCTGTTACAGACCTAGTTACTGGGAATTCATTACTTGCCAT
GGACTACCTTTGCTAAGAAAAGTCTGAATGAGAAGATGGCAGGACGTCTGAAAAAAAAAAGTTATAATTAA
TAAAATCTGCGGAGAATTGTAAAAAAAAAAAAAAAAAA
```

GenBank

```
LOCUS       AJ223353                808 bp    mRNA    linear   PRI 07-OCT-2008
DEFINITION  Homo sapiens mRNA for histone H2B, clone pJG4-5-15.
ACCESSION   AJ223353
VERSION     AJ223353.1  GI:3255998
KEYWORDS    histone H2B.
SOURCE      Homo sapiens (human)
  ORGANISM  Homo sapiens
            Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Euteleostomi;
            Mammalia; Eutheria; Euarchontoglires; Primates; Haplorrhini;
            Catarrhini; Hominidae; Homo.
REFERENCE   1
  AUTHORS   Lorain,S., Quivy,J.P., Monier-Gavelle,F., Scamps,C., Lecluse,Y.,
            Almouzni,G. and Lipinski,M.
  TITLE     Core histones and HIRIP3, a novel histone-binding protein, directly
            interact with WD repeat protein HIRA
  JOURNAL   Mol. Cell. Biol. 18 (9), 5546-5556 (1998)
   PUBMED   9710638
REFERENCE   2  (bases 1 to 808)
  AUTHORS   Lipinski,M.
  TITLE     Direct Submission
  JOURNAL   Submitted (08-JAN-1998) Lipinski M., CNRS URA 1156, Institut
            Gustave Roussy, Rue Camille Desmoulins, 94805, FRANCE
FEATURES             Location/Qualifiers
     source          1..808
                     /organism="Homo sapiens"
                     /mol_type="mRNA"
                     /db_xref="taxon:9606"
                     /clone="pjG4-5-15"
                     /cell_line="HeLa"
                     /tissue_type="cervix carcinoma"
                     /dev_stage="adult"
     gene            1..808
                     /gene="HIRIP2"
     CDS             29..409
                     /gene="HIRIP2"
                     /codon_start=1
                     /product="Histone H2B"
                     /protein_id="CAA11277.1"
                     /db_xref="GI:3255999"
                     /db_xref="GOA:P58876"
                     /db_xref="HGNC:HGNC:4747"
                     /db_xref="InterPro:IPR000558"
                     /db_xref="InterPro:IPR007125"
                     /db_xref="InterPro:IPR009072"
                     /db_xref="UniProtKB/Swiss-Prot:P58876"
                     /translation="MPEPTKSAPAPKKGSKKAVTKAQKKDGKKRKRSRKESYSVYVYK
                     VLKQVHPDTGISSKAMGIMNSFVNDIFERIAGEASRLAHYNKRSTITSREIQTAVRLL
                     LPGELAKHAVSEGTKAVTKYTSSK"
     regulatory      769..774
                     /regulatory_class="polyA_signal_sequence"
                     /gene="HIRIP2"
ORIGIN
        1 ggggagtgtg ctaactatta acgctacgat gcctgaacct accaagtctg ctcctgcccc
       61 aaagaagggc tccaagaagg cggtgactaa ggctcagaag aaggacggga agaagcgcaa
      121 gcgcagccgc aaggagagct attcagtgta tgtgtacaag gtgctgaagc aggtccatcc
      181 cgacaccggc atctcttcca aggcaatggg gatcatgaat tccttcgtca acgacatctt
      241 cgagcgcatc gcaggcgagg cttcccgcct ggcgcattac aacaagcgct cgaccatcac
      301 ctccagggag atccagacgg ccgtgcgcct gctgcttccg ggggagctgg ccaagcacgc
      361 cgtgtcggag ggcaccaagg ccgtcaccaa gtacaccagt tccaagtaac tttgccaagg
      421 gagagacatg aagacagagg agaaatgaat gcataaaata actgataata tgaatctata
      481 catagaactt aggaagtctc atctgcctga aaatgactgt gtggatccca cccaaatcca
      541 actcatcctg gtttgctgca cactggttca tcaaaagaag gttaccgagg ggaaggaact
      601 aaaggtgttt gcacttcatg ttacttttg agtttataaa cataaaaaca gaatttactt
      661 ctgttacaga cctagttact gggaattcat tacttgccat ggactacctt tgctaagaaa
      721 agtctgaatg agaagatggc aggacgtctg aaaaaaaaag ttataattaa taaaatctgc
```

```
781 ggagaattgt aaaaaaaaaa aaaaaaaa
//
```

BLAST output

```
# BLASTX 2.2.28 +
# Query: gi|322830704:1426-2962 Boreus elegans mitochondrion, complete genome
# Database: nem
# Fields: query id, subject id, % identity, alignment length, mismatches, gap opens, q. start, q. end, s. start
# 405 hits found
gi|322830704:1426-2962  gi|225622197|ref|YP_002725710.1|        61.71   444     169     1       4       1335
gi|322830704:1426-2962  gi|288903410|ref|YP_003434131.1|        61.99   442     167     1       10      1335
gi|322830704:1426-2962  gi|326536058|ref|YP_004300493.1|        62.44   442     165     1       10      1335
...
```

FASTQ

Illumina's fastq-files-explained.html

```
@K00114:84:H2N7KBBXX:8:1101:6105:1457 1:N:0
ATTTCCTTTTCGTGGATACGACANAGGATGATGCAAGCAGCGAGTAAACGATTTGATGAGGCTTAGCGCATCATGAATAGCACAGATGATGTGGAGAAAG
+
AAFFFKFFKKKKKKKKKKKFKKK#KFKFFFAKKKKF7,AFAKAAKKKKKK,FAFFFKA7FFKKFK,,7AFFKAKKKKKF,7FKK7AFFF,A,,,7AFKF7
@K00114:84:H2N7KBBXX:8:1101:6856:1457 1:N:0
TGAAGAAATAAAACAAGTAACAGNGACACCAACTCAGCTATCAGCCGCTTCCGTGATTAGACGCAGCCGTTCAACGGCCATTGCGAGCAAGGCAAATGTA
+
AAFFFKKKKKKKKKKKKKKKKKK#KKKKKKKKKKKKKKKKKFKFFAFFKKKKKKKKKKKKFKKFFAF,(7AFAFFKFFKFFKKKKKK7FKKKKKKKKA,A
...
```

SAM (Sequence Alignment/Map)
BAM (compressed Binary version of SAM output)

see https://samtools.github.io/hts-specs/SAMv1.pdf for the full specifications

```
@HD VN:1.5 SO:coordinate
@SQ SN:ref LN:45
r001   99 ref  7 30 8M2I4M1D3M = 37  39 TTAGATAAAGGATACTG *
r002    0 ref  9 30 3S6M1P1I4M *  0   0 AAAAGATAAGGATA    *
r003    0 ref  9 30 5S6M       *  0   0 GCCTAAGCTAA       * SA:Z:ref,29,-,6H5M,17,0;
...
```

VCF

(variant call format; http://samtools.github.io/hts-specs)

```
##fileformat=VCFv4.0
##fileDate=20090805
##source=myImputationProgramV3.1
##reference=1000GenomesPilot-NCBI36
##phasing=partial
##INFO=<ID=NS,Number=1,Type=Integer,Description="Number of Samples With Data">
##INFO=<ID=DP,Number=1,Type=Integer,Description="Total Depth">
##INFO=<ID=AF,Number=.,Type=Float,Description="Allele Frequency">
##INFO=<ID=AA,Number=1,Type=String,Description="Ancestral Allele">
##INFO=<ID=DB,Number=0,Type=Flag,Description="dbSNP membership, build 129">
##INFO=<ID=H2,Number=0,Type=Flag,Description="HapMap2 membership">
##FILTER=<ID=q10,Description="Quality below 10">
##FILTER=<ID=s50,Description="Less than 50% of samples have data">
##FORMAT=<ID=GT,Number=1,Type=String,Description="Genotype">
##FORMAT=<ID=GQ,Number=1,Type=Integer,Description="Genotype Quality">
##FORMAT=<ID=DP,Number=1,Type=Integer,Description="Read Depth">
##FORMAT=<ID=HQ,Number=2,Type=Integer,Description="Haplotype Quality">
#CHROM POS     ID        REF ALT    QUAL FILTER INFO                              FORMAT      NA00001        NA
20     14370   rs6054257 G      A       29   PASS   NS=3;DP=14;AF=0.5;DB;H2           GT:GQ:DP:HQ 0|0:48:1:51,5
20     17330   .         T      A       3    q10    NS=3;DP=11;AF=0.017               GT:GQ:DP:HQ 0|0:49:3:58,5
20     1110696 rs6040355 A      G,T     67   PASS   NS=2;DP=10;AF=0.333,0.667;AA=T;DB GT:GQ:DP:HQ 1|2:21:6:23,2
20     1230237 .         T      .       47   PASS   NS=3;DP=13;AA=T                   GT:GQ:DP:HQ 0|0:54:7:56,6
20     1234567 microsat1 GTCT   G,GTACT 50   PASS   NS=3;DP=9;AA=G                    GT:GQ:DP    0/1:35:4
...
```

Often, we need to take a file and tweak its format for existing tools (e.g. some FASTA headers can be "strange").

Other times we need to write a programme/script that will either create input for, or accept output from, other tools.

Today we will cover a lot of the Python "basics", for example:

- reading text
- processing lines in a file
- creating new files
- appending and writing data to files
- renaming
- moving
- copying
- deleting
- doing "stuff" to each file in a folder/directory

Getting data from a file

Getting data "out" of a file is a **three step** process:

1. open the file (by making a connection)
2. read the file contents (through this connection)
3. close the file (connection)

`open()` is a function that takes a string argument (the name of the file) and returns a **File object**.

File objects are a new type of data that represent a file on disk. They have useful methods, like strings, but unlike strings we can't really print them!

```
# Start python3 on the server
p3 # or python3 or python
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
# Open a pre-existing file
open("my_DNA.txt")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: 'my_DNA.txt'
```

```
# Try again, open the file (correct name this time!)
open("my_dna.txt")
<_io.TextIOWrapper name='my_dna.txt' mode='r' encoding='UTF-8'>
```

```
# Open the file and have it as a file object
my_file = open("my_dna.txt")
```

```
# Try printing the file object
print(my_file
# I pressed return before closing the bracket! DOH!
# The three dots are displayed by Python to indicate that
# it is still an incomplete command, until I type the closing )
... )
<_io.TextIOWrapper name='my_dna.txt' mode='r' encoding='UTF-8'>
```

`read()` is a File object method that returns the contents as a string. It has no arguments: just `()`.

```
# Try reading and displaying file object contents
my_file = open("my_dna.txt")
print(my_file.read())
GGGGGGGGG
AAAAAAAAA
TTTTTTTTT
CCCCCCCCC
 [empty line]
```

The lines above look this way because of the (invisible) `\n` "new line" character at the end of each line.

Note that we have an extra line at the bottom.

If we HAD done this slightly differently, without the `print` function, we would see the new line characters actually in the contents, the "representation" that the interpreter sees:

```
my_file.read()
'GGGGGGGGG\nAAAAAAAAA\nTTTTTTTTT\nCCCCCCCCC\n'
```

```
# read() only works once: something to watch out for!
```

```
# Make a connection
my_file = open("my_dna.txt")
# Store the file contents in a variable called my_file_contents
my_file_contents = my_file.read()
# Show what is there
my_file_contents
'GGGGGGGGG\nAAAAAAAAA\nTTTTTTTTT\nCCCCCCCCC\n'
```

```
# Connection has been used
my_file.read()
''
# We still have the file contents stored though
my_file_contents
'GGGGGGGGG\nAAAAAAAAA\nTTTTTTTTT\nCCCCCCCCC\n'
```

Typically, we want to remove the blank line at the end of the file object, using the `rstrip()` method:

```
# Read and strip the new line characters from a file object
my_file = open("my_dna.txt")
my_file_contents = my_file.read()
```

```
# Remove the "newline" from the end of the file contents
my_dna1 = my_file_contents.rstrip("\n")
print(my_dna1)
GGGGGGGGG
AAAAAAAAA
TTTTTTTTT
CCCCCCCCC
```

Notice how this version doesn't have the extra empty line at the end, but must still have the new line characters "between the lines" we see on the screen.

```
# So is our file object "connection" still open?
my_file.closed
```

```
False
```

```
my_file.close()
my_file.closed
```

```
True
```

For what we are doing today, we would actually like to have all of our sequence on a single line, so luckily, there are a few ways we can fix this.

```
# Remove all the "newline" characters from the file contents using replace()
# Remember that replace() is expecting from this, to this to be specified
my_file_contents.replace("\n",)
```

```
Traceback (most recent call last):
  File "", line 1, in
TypeError: replace expected at least 2 arguments, got 1
```

```
my_dna2 = my_file_contents.replace("\n","")
print(my_dna2)
```

```
GGGGGGGGGAAAAAAAAATTTTTTTTTCCCCCCCCC
```

```
# Hasnt changed the original variable value
my_file_contents
```

```
'GGGGGGGGG\nAAAAAAAAA\nTTTTTTTTT\nCCCCCCCCC\n'
```

```
# We could also remove all the newlines from the file contents using split()
# split() has space and newline as default separators (but we can specify different things)
my_dna3 = my_file_contents.split()
print(my_dna3)
```

```
['GGGGGGGGG', 'AAAAAAAAA', 'TTTTTTTTT', 'CCCCCCCCC']
```

```
my_dna3
```

```
['GGGGGGGGG', 'AAAAAAAAA', 'TTTTTTTTT', 'CCCCCCCCC']
```

As you will have noticed, the output from `split()` produces a different kind of data type, a **list**, which has square brackets `[]`. We will be learning how to use lists later...

Writing to files

This is similar to what we did before when opening a file. It is also **three steps**:

1. open a connection
2. write/output through the connection
3. close the connection

However, to write to a file we have to use a second argument to `open`:

```
my_outfile = open("out.txt", "w")
```

w stands for "write". Once we have opened a file for writing, we can use the `write()` method:

```
# Create an output file
```

```
my_outfile = open("out.txt", "w")
```

```
  File "<stdin>", line 1
    my_outfile = open("out.txt", "w")
    ^
IndentationError: unexpected indent
```

```
# Try again, this time without a leading space...
```

```
my_outfile = open("out.txt", "w")
```

```
# Write some data to the output file
# It will NOT have EOL (end of line) characters unless we tell it to
```

```
my_outfile.write("Today is Friday.")
```

```
16
```

```
# What does the 16 refer to?
len("Today is Friday.")
16
```

```
# Check that this has worked
my_outfile
<_io.TextIOWrapper name='out.txt' mode='w' encoding='UTF-8'>
print(my_outfile)
<_io.TextIOWrapper name='out.txt' mode='w' encoding='UTF-8'>
```

We could try to open the file in our favourite text editor....but....

Once we've finished writing data to a file, we **have** to close it!

```
# Close the file
my_outfile.close()
```

```
# Check to see if it has worked by just opening and reading the contents
open("out.txt").read()
'Today is Friday.'
```

```
# Re-open the file connection, but this time, to append to it
my_outfile = open("out.txt", "a")
my_outfile
<_io.TextIOWrapper name='out.txt' mode='a' encoding='UTF-8'>
```

```
# Write some data to the output file, with a new line character
my_outfile.write("Still Friday." + "\n")
14
```

```
# Write some more data to the output file, with a horizontal tab character
my_outfile.write("\t" + "and tomorrow, if the Gods are willing, it will be Saturday....")
63
```

```
# We can't view the new contents of the file until it is closed!
# The original file is still there...!
open("out.txt").read()
```

```
'Today is Friday.'
```

```
# We will not see the other text we "saved"
# until the file is closed
my_outfile.close()
```

```
# View the file contents again
open("out.txt").read()
```

```
'Today is Friday.Still Friday.\n\tand tomorrow, if the Gods are willing, it will be Saturd
```

```
print(open("out.txt").read())
```

```
Today is Friday.Still Friday.
        and tomorrow, if the Gods are willing, it will be Saturday....
```

The `read()` bit worked this second time because we had done `open()` again first.

I dont want to have to remember to close things....

There is another, safer, way to use file objects. This involves the use of the word `with` as part of the command.

```
# Original way to do it
my_file = open("my_dna.txt")
my_file_contents = my_file.read()
my_file.closed
False
my_file.close()
```

```
# Better way to do it when reading
with open("my_dna.txt") as my_file:
...     my_file_contents = my_file.read()
... my_file_contents
  File "", line 3
    my_file_contents
    ^
SyntaxError: invalid syntax
```

```
my_file.closed
False
```

```
# Try again with correct indentations (all the same) this time...
with open("my_dna.txt") as my_file:
       my_file_contents = my_file.read()
       my_file_contents
...
'GGGGGGGGG\nAAAAAAAAA\nTTTTTTTTT\nCCCCCCCCC\n'
```

```
my_file.closed
True
```

```
# Better way to do it when writing
with open("my_dna.txt", "w") as my_file:
     my_file.write("\nSome additional text")
```

```
...
```

```
21
```

```
my_file.closed
```

```
True
```

```
# Check all is OK...
```

```
open("my_dna.txt").read()
```

```
'\nSome additional text'
```

```
print(open("my_dna.txt").read())
```

```

```

```
Some additional text
```

```
CTRL-D to exit
```

Quick summary of things

Name	Action	Argument	Returns	Type
open()	opens a file connection for reading or writing	filename, optional mode (both strings)	File object	built in function
read()	reads the contents of a file connection	none	string	method of File objects
rstrip()	removes characters from end of string (usually newline)	string to remove	string	method of string objects
write()	writes text to a file connection	string to write	string	method of File objects
close()	closes a file connection	none	nothing	method of File objects

---

Working with files

File "paths" are the strings that we use to describe (to Python as well!) where to find the files that we want.

In Python (and Linux/bash, as we know already), characters preceded by `\` often have a special meaning or are **escaped** characters. To avoid this, we use `/` instead:

```
c:/path/to/a/file
```

On Mac/Linux/Unix, we use the `/` character as normal:

```
/localdisk/home/aivens2/OnLineExamStuff/all_exam_questions.txt
```

Basic file manipulation

"Modules" are essentially pre-written files containing Python definitions and statements.

They are only available to us if we `import` them into the python interpreter, our python "bubble".

Functions for file manipulation within Python live in a module called the **os** module.

```
# In Linux, find out where we are, after changing to our home space
cd # or cd ~ or cd ${HOME}
echo ${PWD}
/localdisk/home/aivens2
```

```
# List the contents of the directory Exercises/Lecture06
ls ~/Exercises/Lecture06
```

```
AEIinSubjectAcc.starts.exercise.out  HSPscore.150.exercise.out           nem.fasta  nem.ps
al_leng_pcID.exercise.out            HSPscore.250.exercise.out           nem.pdb    nem.pt
blastoutput1.out                     HSPscore.350.exercise.out           nem.phr    nem.pt
blastoutput2.out                     HSPscore.morethan.350.exercise.out  nem.pin    run_th
Fewer.than20MM.exercise.out          link_testsequence.fasta             nem.pjs    saved_
first_seq_character.txt              Mismatchpercent.exercise.out        nem.pot    Subjec
```

```
# Let's try and be tidy...(think git)
# Make a directory for today, let's call it Lecture12
mkdir ~Exercises/Lecture12
# List the contents of the directory Lecture12
ls ~/Exercises/Lecture12
```

```
# Note that we are still in our home space...
```

```
# Start python3
```

```
python3 # or p3 or python
```

```
Python 3.12.3 (main, Aug 14 2025, 17:47:21) [GCC 13.3.0] on linux
Type "help", "copyright", "credits" or "license" for more information.
>>>
```

```
# Need to be able to "talk" to the operating system (os)
```

```
# We have the os module to do that
```

```
import os
```

```
# Is it just a command to run?
```

```
os
```

```
<module 'os' (frozen)'>
```

```
# Is there any (useful) help?
```

```
help(os)
```

```
Help on module os:

NAME
    os - OS routines for NT or Posix depending on what system we're on.

MODULE REFERENCE
    https://docs.python.org/3.12/library/os.html

    The following documentation is automatically generated from the Python
    source files.  It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations.  When in doubt, consult the module reference at the
    location listed above.

DESCRIPTION
    This exports:
      - all functions from posix or nt, e.g. unlink, stat, etc.
      - os.path is either posixpath or ntpath
      - os.name is either 'posix' or 'nt'
      - os.curdir is a string representing the current directory (always '.')
      - os.pardir is a string representing the parent directory (always '..')
      - os.sep is the (or a most common) pathname separator ('/' or '\\')
      - os.extsep is the extension separator (always '.')
      - os.altsep is the alternate pathname separator (None or '/')
      - os.pathsep is the component separator used in $PATH etc
      - os.linesep is the line separator in text files ('\r' or '\n' or '\r\n')
      - os.defpath is the default search path for executables
      - os.devnull is the file path of the null device ('/dev/null', etc.)

    Programs that import and use 'os' stand a better chance of being
    portable between different platforms.  Of course, they must then
    only use functions that are defined by all platforms (e.g., unlink
    and opendir), and leave all pathname manipulation to os.path
    (e.g., split and join).

CLASSES
    builtins.Exception(builtins.BaseException)
```

```
        builtins.OSError
    builtins.object
        posix.DirEntry
    builtins.tuple(builtins.object)
        stat_result
        statvfs_result
        terminal_size
        posix.sched_param
        posix.times_result
        posix.uname_result
        posix.waitid_result
...
```

```
# We access "things" in a module using the . then whatever we want to access.
```

```
# What can we access, if we aren't sure what we want?
```

```
# Type the module name, then the dot, then the TAB key twice...
```

```
os.
```

```
Display all 384 possibilities? (y or n)
os.CLD_CONTINUED                os.O_FSYNC                      os.WEXITSTATUS(                 os.geteuid()                    os.sched_param(
os.CLD_DUMPED                   os.O_LARGEFILE                  os.WIFCONTINUED(                os.getgid()                     os.sched_rr_get_interval(
os.CLD_EXITED                   os.O_NDELAY                     os.WIFEXITED(                   os.getgrouplist(                os.sched_setaffinity(
os.CLD_KILLED                   os.O_NOATIME                    os.WIFSIGNALED(                 os.getgroups()                  os.sched_setparam(
os.CLD_STOPPED                  os.O_NOCTTY                     os.WIFSTOPPED(                  os.getloadavg()                 os.sched_setscheduler(
os.CLD_TRAPPED                  os.O_NOFOLLOW                   os.WNOHANG                      os.getlogin()                   os.sched_yield()
os.CLONE_FILES                  os.O_NONBLOCK                   os.WNOWAIT                      os.getpgid(                     os.sendfile(
os.CLONE_FS                     os.O_PATH                       os.WSTOPPED                     os.getpgrp()                    os.sep
os.CLONE_NEWCGROUP              os.O_RDONLY                     os.WSTOPSIG(                    os.getpid()                     os.set_blocking(
os.CLONE_NEWIPC                 os.O_RDWR                       os.WTERMSIG(                    os.getppid()                    os.set_inheritable(
os.CLONE_NEWNET                 os.O_RSYNC                      os.WUNTRACED                    os.getpriority(                 os.setegid(
os.CLONE_NEWNS                  os.O_SYNC                       os.W_OK                         os.getrandom(                   os.seteuid(
os.CLONE_NEWPID                 os.O_TMPFILE                    os.XATTR_CREATE                 os.getresgid()                  os.setgid(
os.CLONE_NEWTIME                os.O_TRUNC                      os.XATTR_REPLACE                os.getresuid()                  os.setgroups(
os.CLONE_NEWUSER                os.O_WRONLY                     os.XATTR_SIZE_MAX               os.getsid(                      os.setns(
os.CLONE_NEWUTS                 os.POSIX_FADV_DONTNEED          os.X_OK                         os.getuid()                     os.setpgid(
os.CLONE_SIGHAND                os.POSIX_FADV_NOREUSE           os.abc                          os.getxattr(                    os.setpgrp()
os.CLONE_SYSVSEM                os.POSIX_FADV_NORMAL            os.abort()                      os.initgroups(                  os.setpriority(
os.CLONE_THREAD                 os.POSIX_FADV_RANDOM            os.access(                      os.isatty(                      os.setregid(
os.CLONE_VM                     os.POSIX_FADV_SEQUENTIAL        os.altsep                       os.kill(                        os.setresgid(
os.DirEntry()                   os.POSIX_FADV_WILLNEED          os.chdir(                       os.killpg(                      os.setresuid(
os.EFD_CLOEXEC                  os.POSIX_SPAWN_CLOSE            os.chmod(                       os.lchown(                      os.setreuid(
os.EFD_NONBLOCK                 os.POSIX_SPAWN_DUP2             os.chown(                       os.linesep                      os.setsid()
os.EFD_SEMAPHORE                os.POSIX_SPAWN_OPEN             os.chroot(                      os.link(                        os.setuid(
os.EX_CANTCREAT                 os.PRIO_PGRP                    os.close(                       os.listdir(                     os.setxattr(
os.EX_CONFIG                    os.PRIO_PROCESS                 os.closerange(                  os.listxattr(                   os.spawnl(
os.EX_DATAERR                   os.PRIO_USER                    os.confstr(                     os.lockf(                       os.spawnle(
os.EX_IOERR                     os.P_ALL                        os.confstr_names                os.login_tty(                   os.spawnlp(
os.EX_NOHOST                    os.P_NOWAIT                     os.copy_file_range(             os.lseek(                       os.spawnlpe(
os.EX_NOINPUT                   os.P_NOWAITO                    os.cpu_count()                  os.lstat(                       os.spawnv(
os.EX_NOPERM                    os.P_PGID                       os.ctermid()                    os.major(                       os.spawnve(
os.EX_NOUSER                    os.P_PID                        os.curdir                       os.makedev(                     os.spawnvp(
os.EX_OK                        os.P_PIDFD                      os.defpath                      os.makedirs(                    os.spawnvpe(
os.EX_OSERR                     os.P_WAIT                       os.device_encoding(             os.memfd_create(                os.splice(
os.EX_OSFILE                    os.PathLike()                   os.devnull                      os.minor(                       os.st
os.EX_PROTOCOL                  os.RTLD_DEEPBIND                os.dup(                         os.mkdir(                       os.stat(
os.EX_SOFTWARE                  os.RTLD_GLOBAL                  os.dup2(                        os.mkfifo(                      os.stat_result(
os.EX_TEMPFAIL                  os.RTLD_LAZY                    os.environ                      os.mknod(                       os.statvfs(
os.EX_UNAVAILABLE               os.RTLD_LOCAL                   os.environb                     os.name                         os.statvfs_result(
os.EX_USAGE                     os.RTLD_NODELETE                os.error(                       os.nice(                        os.strerror(
os.F_LOCK                       os.RTLD_NOLOAD                  os.eventfd(                     os.open(                        os.supports_bytes_environ
os.F_OK                         os.RTLD_NOW                     os.eventfd_read(                os.openpty()                    os.supports_dir_fd
os.F_TEST                       os.RWF_APPEND                   os.eventfd_write(               os.pardir                       os.supports_effective_ids
os.F_TLOCK                      os.RWF_DSYNC                    os.execl(                       os.path                         os.supports_fd
os.F_ULOCK                      os.RWF_HIPRI                    os.execle(                      os.pathconf(                    os.supports_follow_symlinks
os.GRND_NONBLOCK                os.RWF_NOWAIT                   os.execlp(                      os.pathconf_names               os.symlink(
os.GRND_RANDOM                  os.RWF_SYNC                     os.execlpe(                     os.pathsep                      os.sync()
os.GenericAlias(                os.R_OK                         os.execv(                       os.pidfd_open(                  os.sys
os.MFD_ALLOW_SEALING            os.SCHED_BATCH                  os.execve(                      os.pipe()                       os.sysconf(
os.MFD_CLOEXEC                  os.SCHED_FIFO                   os.execvp(                      os.pipe2(                       os.sysconf_names
os.MFD_HUGETLB                  os.SCHED_IDLE                   os.execvpe(                     os.popen(                       os.system(
os.MFD_HUGE_16GB                os.SCHED_OTHER                  os.extsep                       os.posix_fadvise(               os.tcgetpgrp(
os.MFD_HUGE_16MB                os.SCHED_RESET_ON_FORK          os.fchdir(                      os.posix_fallocate(             os.tcsetpgrp(
os.MFD_HUGE_1GB                 os.SCHED_RR                     os.fchmod(                      os.posix_spawn(                 os.terminal_size(
os.MFD_HUGE_1MB                 os.SEEK_CUR                     os.fchown(                      os.posix_spawnp(                os.times()
os.MFD_HUGE_256MB               os.SEEK_DATA                    os.fdatasync(                   os.pread(                       os.times_result(
os.MFD_HUGE_2GB                 os.SEEK_END                     os.fdopen(                      os.preadv(                      os.truncate(
os.MFD_HUGE_2MB                 os.SEEK_HOLE                    os.fork()                       os.putenv(                      os.ttyname(
os.MFD_HUGE_32MB                os.SEEK_SET                     os.forkpty()                    os.pwrite(                      os.umask(
os.MFD_HUGE_512KB               os.SPLICE_F_MORE                os.fpathconf(                   os.pwritev(                     os.uname()
os.MFD_HUGE_512MB               os.SPLICE_F_MOVE                os.fsdecode(                    os.read(                        os.uname_result(
os.MFD_HUGE_64KB                os.SPLICE_F_NONBLOCK            os.fsencode(                    os.readlink(                    os.unlink(
os.MFD_HUGE_8MB                 os.ST_APPEND                    os.fspath(                      os.readv(                       os.unsetenv(
os.MFD_HUGE_MASK                os.ST_MANDLOCK                  os.fstat(                       os.register_at_fork(            os.unshare(
os.MFD_HUGE_SHIFT               os.ST_NOATIME                   os.fstatvfs(                    os.remove(                      os.urandom(
os.Mapping()                    os.ST_NODEV                     os.fsync(                       os.removedirs(                  os.utime(
os.MutableMapping()             os.ST_NODIRATIME                os.ftruncate(                   os.removexattr(                 os.wait()
os.NGROUPS_MAX                  os.ST_NOEXEC                    os.fwalk(                       os.rename(                      os.wait3(
os.O_ACCMODE                    os.ST_NOSUID                    os.get_blocking(                os.renames(                     os.wait4(
os.O_APPEND                     os.ST_RDONLY                    os.get_exec_path(               os.replace(                     os.waitid(
os.O_ASYNC                      os.ST_RELATIME                  os.get_inheritable(             os.rmdir(                       os.waitid_result(
```

```
os.O_CLOEXEC            os.ST_SYNCHRONOUS       os.get_terminal_size(   os.scandir(                os.waitpid(
os.O_CREAT              os.ST_WRITE             os.getcwd()             os.sched_get_priority_max(  os.waitstatus_to_exitcode(
os.O_DIRECT             os.TMP_MAX              os.getcwdb()            os.sched_get_priority_min(  os.walk(
os.O_DIRECTORY          os.WCONTINUED           os.getegid()            os.sched_getaffinity(       os.write(
os.O_DSYNC              os.WCOREDUMP(           os.getenv(              os.sched_getparam(          os.writev(
os.O_EXCL               os.WEXITED              os.getenvb(             os.sched_getscheduler(
```

```
# Now we can get current working directory, i.e. where are we?
```

```
os.getcwd()
```

```
'/localdisk/home/aivens2'
```

```
# Can we directly use a system variable, e.g. ${ ?
```

```
os.chdir("${HOME}")
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2] No such file or directory: '${HOME}'
```

```
os.chdir(os.environ['HOME'])
```

```
os.getcwd()
```

```
'/localdisk/home/aivens2'
```

```
# We could store this in a variable if we wanted!
```

```
myhome = os.environ['HOME']
```

```
print("My homespace is",myhome)
```

```
My homespace is /home/aivens2
```

```
# Now, change directory to ~/Exercises/Lecture06
```

```
# This works
```

```
os.chdir("Exercises/Lecture06")
```

```
# This is "safer", as it is the full path
```

```
os.chdir("/localdisk/home/aivens2/Exercises/Lecture06")
```

```
# Now we can check our current working directory again
```

```
os.getcwd()
```

```
'/localdisk/home/aivens2/Exercises/Lecture06'
```

```
# We could store this in a variable if we wanted!
```

```
where_i_am_now = os.getcwd()
```

```
print("I'm now in the folder",where_i_am_now)
```

```
I'm now in the folder /localdisk/home/aivens2/Exercises/Lecture06
```

```
# Would this give the same result? No...
```

```
os.environ['PWD']
```

```
'/home/aivens2'
```

... because the `os.environ()` variables are what they were when we started our python session

START OF BRIEF DETOUR.... we learn about dictionaries later, I don't expect you to know about these yet....!

```
# All the environment values are held in a dictionary that we can view
for key, value in os.environ.items():
     print('{}: {}'.format(key, value))
```

```
SHELL: /bin/bash
PICARD: /localdisk/home/ubuntu-software/picard/picard.jar
NCBI_API_KEY: f8230825b624062e53bb5ab7.....
mypersonalGitHub_username: ......
mypersonalgittoken: ......
LANGUAGE: en_GB:en
PWD: /home/aivens2
KRB5CCNAME: FILE:/tmp/krb5cc_864524_QuZf6j
LOGNAME: aivens2
XDG_SESSION_TYPE: tty
MOTD_SHOWN: pam
HOME: /home/aivens2
LANG: en_GB.UTF-8
SSH_CONNECTION: 10.65.128.61 62371 129.215.237.85 22
XDG_SESSION_CLASS: user
TERM: xterm
USER: aivens2
DISPLAY: localhost:12.0
SHLVL: 1
XDG_SESSION_ID: 12163
XDG_RUNTIME_DIR: /run/user/864524
SSH_CLIENT: 10.65.128.61 62371 22
XDG_DATA_DIRS: /usr/local/share:/usr/share:/var/lib/snapd/desktop
PATH: /localdisk/home/ubuntu-software/ncbi-blast-2.13.0+-src/bin: ...
DBUS_SESSION_BUS_ADDRESS: unix:path=/run/user/864524/bus
SSH_TTY: /dev/pts/5
_: /usr/bin/python3
OLDPWD: /home/aivens2/Exercises/Lecture12
```

END OF BRIEF DETOUR.... more in other lectures....

```
# List the contents of the current directory
NOTE that yours might be different depending on how many of the exercises you did for Lecture 06!
```

```
os.listdir()
```

```
['nem.fasta', 'first_seq_character.txt', 'nem.pdb', 'nem.ptf', 'nem.pin', 'nem.phr',
  'nem.psq', 'nem.pot', 'nem.pto', 'nem.pjs', 'testsequence.fasta', 'link_testsequence.fas
  'saved_testsequence.fasta', 'blastoutput1.out', 'blastoutput2.out', 'run_this_script_v1'
  'Subject_accessions.exercise.out', 'al_leng_pcID.exercise.out', 'Top10.HSPs.exercise.out
```

```
  'Mismatchpercent.exercise.out', 'HSPscore.morethan.350.exercise.out', 'HSPscore.350.exer
  'HSPscore.250.exercise.out', 'HSPscore.150.exercise.out', 'AEIinSubjectAcc.starts.exerci
'Fewer.than20MM.exercise.out']
```

```
# List the contents of the current interactive "bubble"
# What does yours say?
dir()
```

```
# How many files are there?
len(os.listdir())
26
```

```
# It's a list data type, so we can use an index to access the information
# This is python, so the index counting starts at 0, NOT 1...!
os.listdir()[0]
'nem.fasta'
os.listdir()[1]
'first_seq_character.txt'
```

```
# Listing more than a couple of files at a time can
# be sub-optimal for viewing...
os.listdir()[0:7]
['nem.fasta', 'first_seq_character.txt', 'nem.pdb', 'nem.ptf', 'nem.pin',
'nem.phr', 'nem.psq']
```

```
# ... so we can separate the list elements with a
# new line \n, and print them
print( "\n" . join ( os.listdir() [0:7] ) )
```

or

```
print("\n".join(os.listdir()[0:7]))
nem.fasta
first_seq_character.txt
nem.pdb
nem.ptf
nem.pin
nem.phr
nem.psq
```

```
# ...and much nicer when sorted too
```

```
print("\n".join(sorted(os.listdir()[0:7])))
```

```
first_seq_character.txt
nem.fasta
nem.pdb
nem.phr
nem.pin
nem.psq
nem.ptf
```

Renaming/moving files, *etc.*

Don't forget, "renaming" is the same as "moving", moving a file from one name to another name, or from one place to another. Could be done in the same folder, or in different folders...

```
# This is a straightforward process: these are just example commands
os.rename("old.txt", "new.txt")
os.rename("biology/old.txt", "biology/new.txt")
os.rename("old_directory", "new_directory")
```

Making a new directory (folder)

```
# Make a directory (implicitly in your current working directory)
# In other words, this assumes that you know where you are!
os.mkdir("MyNewDir")
```

```
# It will tell you if it is already there
os.mkdir("MyNewDir")
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileExistsError: [Errno 17] File exists: 'MyNewDir'
```

```
# Make a directory in an explicitly named location
os.mkdir("/localdisk/home/aivens2/Exercises/Lecture127/MyNewDir")
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2]
No such file or directory:
    '/localdisk/home/aivens2/Exercises/Lecture127/MyNewDir'
```

```
# Make a directory in an explicitly, and correctly, named place
# Note how this time, we used a variable and a string!
os.mkdir(myhome + "/Exercises/Lecture12/MyNewDir/")
```

```
# We can make nested directores (like mkdir -p in bash), but need to use makedirs
os.makedirs("foop/boop/bar")
```

```
# Did it work? We can look using a system call
os.system("ls -alRt foop")
```

```
foop:
total 4
```

```
drwxr-xr-x 1 aivens2 g_aivens2  76 Oct 23 13:17 boop
drwxr-xr-x 1 aivens2 g_aivens2  76 Oct 23 13:17 .
drwxr-xr-x 1 aivens2 g_aivens2 908 Oct 23 13:17 ..

foop/boop:
total 0
drwxr-xr-x 1 aivens2 g_aivens2 76 Oct 23 13:17 .
drwxr-xr-x 1 aivens2 g_aivens2  0 Oct 23 13:17 bar
drwxr-xr-x 1 aivens2 g_aivens2 76 Oct 23 13:17 ..

foop/boop/bar:
total 0
drwxr-xr-x 1 aivens2 g_aivens2 76 Oct 23 13:17 ..
drwxr-xr-x 1 aivens2 g_aivens2  0 Oct 23 13:17 .
0
```

What was that last 0 all about? It's called the exit status.

If it a zero, that means zero errors! See the link for other details (there's lots....).

```
# Can we make the same nested directories again?
os.makedirs("foop/boop/bar")
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File ""/usr/lib/python3.8/os.py", line 223, in makedirs
    mkdir(name, mode)
FileExistsError: [Errno 17] File exists: 'foop/boop/bar'
```

```
# Can we attempt to make the same nested directories again?
os.makedirs("foop/boop/bar", exist_ok=True)
```

Copying and directory trees

For more advanced manipulations, we use the **sh**ell **util**ity **shutil** module.

```
# Import the module
import shutil
```

```
# Copy a file: general syntax example
shutil.copy("original.txt", "copy.txt")
```

```
# ...is different to copying a directory...
shutil.copytree("original_directory", "copy_directory")
```

```
# We can check if a file or directory exists (this is just an example!!)
examfolder="/OnLineExamStuff/"
ExQs="all_exam_questions.txt"
ExAns="all_exam_answers.txt"

if os.path.exists(myhome + examfolder + ExQs):
   open(myhome + examfolder + ExAns).read().rstrip()
```

....

Deleting stuff...

Just like in the Linux terminal environment, deleting files can be just as "dangerous" in Python...!

Use different functions in increasing order of "danger".

```
# Let's delete a file (exam files going!)
os.remove(myhome + examfolder + ExQs)

os.remove(myhome + examfolder + ExAns)

```

```
# Definitely gone
os.remove(myhome + examfolder + ExAns)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
FileNotFoundError: [Errno 2]
No such file or directory:
  '/localdisk/home/aivens2/OnLineExamStuff/all_exam_answers.txt'
```

```
# Let's delete an empty directory...
os.rmdir(myhome + "/Desktop/MyGames")

```

```
# Let's delete a directory and all its contents
shutil.rmtree(myhome + examfolder)

```

Be VERY careful how you issue the above command....

Give feedback
Ask for help

Running external programs: doing a "system call"

Sometimes it's helpful to be able to run an exising programme/script (e.g. an analysis tool e.g. BLAST) from **within** a Python programme/script.

If we are in the "python bubble" and wanted brief access to something in the Linux environment, we can do many simple things using this approach, where we put our bash command inside the brackets:
`os.system()`

```
# Find out what day it is using cal
os.system("cal")
```

```
    October 2025
Su Mo Tu We Th Fr Sa
           1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31

0
```

The above starts in the "python bubble", goes out into our Linux system, runs the programme `cal`, shows the results on the screen, and finally comes back into the "python bubble": we still have the python3 interpreter `>>>` prompt.

If we want to do more sophisticated things, this can be achieved using the **subprocess** module.

```
# To run a programme/script and display the output on the screen
# Import the subprocess module
import subprocess

```

What is available in the `subprocess` module?
Type `subprocess` then dot TAB TAB

```
subprocess.
```

```
subprocess.CalledProcessError(  subprocess.SubprocessError(     subprocess.contextlib
subprocess.CompletedProcess(    subprocess.TimeoutExpired(      subprocess.errno
subprocess.DEVNULL              subprocess.builtins             subprocess.fcntl
subprocess.PIPE                 subprocess.call(                subprocess.getoutput(
subprocess.Popen(               subprocess.check_call(          subprocess.getstatusoutput
subprocess.STDOUT               subprocess.check_output(        subprocess.io
```

Run a programme/script with some options

```
subprocess.call("/bin/date +%B")
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "/usr/lib/python3.12/subprocess.py", line 389, in call
    with Popen(*popenargs, **kwargs) as p:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/subprocess.py", line 1026, in __init__
    self._execute_child(args, executable, preexec_fn, close_fds,
  File "/usr/lib/python3.12/subprocess.py", line 1955, in _execute_child
    raise child_exception_type(errno_num, err_msg, err_filename)
FileNotFoundError: [Errno 2] No such file or directory: '/bin/date +%B'
```

Try again, running it in a shell

```
subprocess.call("/bin/date +%B", shell=True)
```

```
October
0
```

To run a programme/script AND capture the output in a string

```
cmd = "/bin/date +%B"
```

```
month = subprocess.check_output(cmd, shell=True)
```

```
month
```

```
b'October\n'
```

What is the `b` there for? The b denotes a byte string:
bytes are the actual data. Strings are an "abstraction".

What do we get if we run `cal` this way?

```
subprocess.check_output("cal")
```

```
b'    October 2025      \nSu Mo Tu We Th Fr Sa  \n          1  2  3  4  \n 5  6  7  8  9 1
```

```
# Can we use print to make it look nicer? Not this way!
```

```
mycal = subprocess.check_output("cal")
```

```
print(mycal)
```

```
b'     October 2025       \nSu Mo Tu We Th Fr Sa  \n          1  2  3  4  \n 5  6  7  8  9 1
```

```
mycal = subprocess.check_output("cal").decode("utf-8")
```

```
print(mycal)
```

```
    October 2025
Su Mo Tu We Th Fr Sa
          1  2  3  4
 5  6  7  8  9 10 11
12 13 14 15 16 17 18
19 20 21 22 23 24 25
26 27 28 29 30 31
```

```
# Common errors: requesting the whole module,
# not the part that we wanted!
```

```
month = subprocess(cmd, shell=True)
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
TypeError: 'module' object is not callable
```

```
# Common errors: not capturing the output,
# keeping the exit status instead!
```

```
month = subprocess.call(cmd, shell=True)
```

```
October
```

```
month
```

```
0
```

Getting user input from within Python

We can get user input interactively, with help from the **sys** module.

Import the `sys` module

```
import sys
```

What is available in the `sys` module: use dot TAB TAB again

```
sys.
```

```
sys.abiflags
sys.activate_stack_trampoline(
sys.addaudithook(
sys.api_version
sys.argv
sys.audit(
sys.base_exec_prefix
sys.base_prefix
sys.breakpointhook(
sys.builtin_module_names
sys.byteorder
sys.call_tracing(
sys.copyright
sys.deactivate_stack_trampoline()
sys.displayhook(
sys.dont_write_bytecode
sys.exc_info()
sys.excepthook(
sys.exception()
sys.exec_prefix
sys.executable
sys.exit(
sys.flags
sys.float_info
sys.float_repr_style
sys.get_asyncgen_hooks()
sys.get_coroutine_origin_tracking_depth()
sys.get_int_max_str_digits()
sys.getallocatedblocks()
sys.getdefaultencoding()
sys.getdlopenflags()
sys.getfilesystemencodeerrors()
sys.getfilesystemencoding()
sys.getprofile()
sys.getrecursionlimit()
sys.getrefcount(
sys.getsizeof(
sys.getswitchinterval()
sys.gettrace()
sys.getunicodeinternedsize()
sys.hash_info
sys.hexversion
sys.implementation
sys.int_info
sys.intern(
sys.is_finalizing()
sys.is_stack_trampoline_active()
sys.last_exc
sys.last_traceback
sys.last_type(
sys.last_value
sys.maxsize
sys.maxunicode
sys.meta_path
sys.modules
sys.monitoring
sys.orig_argv
sys.path
sys.path_hooks
sys.path_importer_cache
sys.platform
sys.platlibdir
sys.prefix
sys.ps1
sys.ps2
sys.pycache_prefix
sys.set_asyncgen_hooks(
sys.set_coroutine_origin_tracking_depth(
sys.set_int_max_str_digits(
sys.setdlopenflags(
sys.setprofile(
sys.setrecursionlimit(
sys.setswitchinterval(
sys.settrace(
sys.stderr
sys.stdin
sys.stdlib_module_names
sys.stdout
sys.thread_info
sys.unraisablehook(
sys.version
sys.version_info
sys.warnoptions
```

What is available in the `sys` module: help(sys)

```
help(sys)
```

```
Help on built-in module sys:

NAME
    sys

MODULE REFERENCE
    https://docs.python.org/3.12/library/sys.html

    The following documentation is automatically generated from the Python
    source files.  It may be incomplete, incorrect or include features that
    are considered implementation detail and may vary between Python
    implementations.  When in doubt, consult the module reference at the
    location listed above.
```

```
DESCRIPTION
    This module provides access to some objects used or maintained by the
    interpreter and to functions that interact strongly with the interpreter.

    Dynamic objects:

    argv -- command line arguments; argv[0] is the script pathname if known
    path -- module search path; path[0] is the script directory, else ''
    modules -- dictionary of loaded modules

    displayhook -- called to show results in an interactive session
    excepthook -- called to handle any uncaught exception other than SystemExit
      To customize printing in an interactive session or to install a custom
      top-level exception handler, assign other functions to replace these.

    stdin -- standard input file object; used by input()
    stdout -- standard output file object; used by print()
    stderr -- standard error object; used for error messages
      By assigning other file objects (or objects that behave like files)
      to these, it is possible to redirect all of the interpreter's I/O.

    last_exc - the last uncaught exception
      Only available in an interactive session after a
      traceback has been printed.
    last_type -- type of last uncaught exception
    last_value -- value of last uncaught exception
    last_traceback -- traceback of last uncaught exception
      These three are the (deprecated) legacy representation of last_exc.

    Static objects:
....
```

```
# Get something from the user!
accession = input("Enter the accession name\n")
```

```
Enter the accession name
AC12345
```

```
# Check whether it has worked
accession
```

```
'AC12345'
```

Getting user input from the Linux command line

Frequently we might be in the Linux environment and have a Python script that we want to run. To do that, we might want to feed it various parameters, e.g. file name, threshold, organism species, whatever....

```
# The command we might want to run in Linux,
# if we had a script with this name!
# ... still thinking about fruit salad!
```

```
# Our MyFruityScript.py script, written using
# a text editor, saved somewhere sensible
# would need to have at least the following lines
```

```
# shebang line
#!/usr/bin/python3
# import modules
import os
import sys
# Now tell Python about the two argument variables
# that will be supplied by the user
# on the command line
first_fruit = sys.argv[1] # it will be apples in this example
second_fruit = sys.argv[2] # it will be lychees in this example
# Output the "result"
print(first_fruit + " are just as nice as " + second_fruit)
```

START OF BRIEF DETOUR to a challenge for the more confident coder...

We could actually write the Python script while we are in the Python "bubble"...

let's try that to get some practice at writing to a connection/pipe....

Note that this is **DEFINITELY NOT** the easiest way to write a script! This is just to show you that it CAN be done... (and it is good practice at doing Python commands!?)

 # Open a connection/pipe to a file called *MyFruityScript.py* and make it writeable

```
fpipe = open("MyFruityScript.py","w")
```

 # Send text "down the fpipe" connection/pipe using `write()`

 # Starting with the shebang line for Python3

```
fpipe.write("#!/usr/bin/python3\n")
fpipe.write("# Script to compare fruit\n")
fpipe.write("# Version 1, 24th Oct 2025\n")
fpipe.write("import os\n")
fpipe.write("import sys\n")
fpipe.write("# Allow for two fruit\n")
fpipe.write("# NOTE_incomplete: should probably include some error traps here for wrong in
fpipe.write("first_fruit  = sys.argv[1]\n")
fpipe.write("second_fruit = sys.argv[2]\n")
fpipe.write("# Standard unchanging answer output\n")
fpipe.write("# NOTE_incomplete: a lot more could be done here...!\n")
fpipe.write("print(first_fruit.upper() + \" are just as nice as  \" + second_fruit.upper()
fpipe.close()
```

os.listdir()

['MyFruityScript.py', 'MyNewDir', 'my_dna.txt']

END OF BRIEF DETOUR!

 # Check to see that the saved script looks OK

print(open("MyFruityScript.py").read())

```
#!/usr/bin/python3
# Script to compare fruit
# Version 1, 24th Oct 2025
import os
import sys
# Allow for two fruit
# NOTE_incomplete: should probably include some error traps here for wrong input etc
first_fruit  = sys.argv[1]
second_fruit = sys.argv[2]
# Standard unchanging answer output
# NOTE_incomplete: a lot more could be done here...!
print(first_fruit.upper() + " are just as nice as " + second_fruit.upper())
```

 # Script looks OK

```
# We could run the saved Python script in a shell from within Python too,
# as long as subprocess has been imported
import subprocess
```

```
# Make the script executable
subprocess.call("chmod 700 MyFruityScript.py", shell=True)
```

```
0
```

```
# Run the script
subprocess.call("./MyFruityScript.py apples lychees", shell=True)
```

```
APPLES are just as nice as LYCHEES
0
```

```
# We are finished now, so let's exit from our Python interpreter "bubble"
exit() # or CTRL-D
```

Now that we have written our Python programme/script, we can also run the script in Linux.
It will call Python3 for us because it has the **shebang** line telling it to use the python3 programme to run the script!

```
# Back where we started...
pwd
/home/aivens2
cd ~/Exercises/Lecture12
ls
my_dna.txt MyFruityScript.py MyNewDir
./MyFruityScript.py apples lychees
APPLES are just as nice as LYCHEES
```

```
# Our code might be good, but the user isn't always sensible...!
./MyFruityScript.py hedgehog lychees
HEDGEHOG are just as nice as LYCHEES
./MyFruityScript.py Arsenal ManCity
ARSENAL are just as nice as MANCITY
```

```
./MyFruityScript.py Starmer Trump
```

```
STARMER are just as nice as TRUMP
```

Interactive version...

```
# We wrote another script called MyFruityScript_v2.py
```

```
cat MyFruityScript_v2.py
```

```
#!/usr/bin/python3
# Script to compare fruit
# Version 2, 24th Oct 2025
import os
import sys
# Allow for two fruit
# NOTE_incomplete: should probably include some error traps here for wrong input etc
# first_fruit  = sys.argv[1] # commented out as not using
first_fruit = input("What is the first fruit?\n\t")
# second_fruit = sys.argv[2] # commented out as not using
second_fruit = input("What is the second fruit?\n\t")
# Standard unchanging answer output
# NOTE_incomplete: a lot more could be done here...!
print(first_fruit.upper() + " are just as nice as " + second_fruit.upper())
```

```
# Lets run it!
```

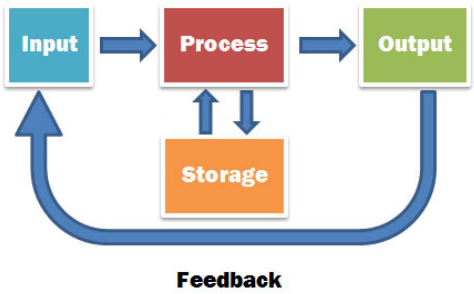
```
chmod 700 *.py && ./MyFruityScript_v2.py
```

```
What is the first fruit?
        pineapple
What is the second fruit?
        dragon fruit
PINEAPPLE are just as nice as DRAGON FRUIT
```

Congratulations! You have just run another awesome Python script!

Exercises

Today's exercises build on everything we have done so far, both Linux-related and Python-related. We will be getting two sequences: one from a remote site, and one from a directory on the server. Both sequences will need some processing in Linux and/or Python; try doing it BOTH ways.



Let's try and be tidy...

Make a directory for today's work, if you haven't done this already, and change directory to it.

```
mkdir ~/Exercises/Lecture12
cd ~/Exercises/Lecture12
```

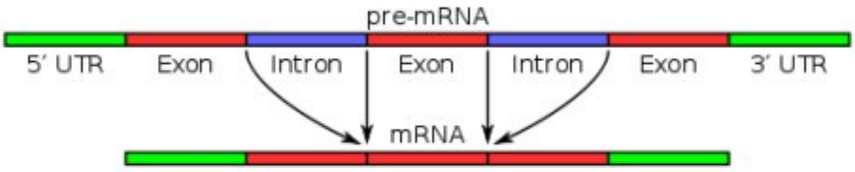
Your tasks

Reminder: a FASTA file stores sequence data and might look like:

```
>sequence_one
cgatcgatcatcgatgcattgtagctatcg
>sequence_two
acagtagctacgtgtgtcgta
```

In the above examples, "sequence_one" and "sequence_two" are the fasta "headers": the ">" character tells us it is the header line.

"Mission One": Make some fasta sequences



1. "local" DNA sequence: take a copy of the file called `plain_genomic_seq.txt`, which is in the `/localdisk/data/BPSM/Lecture12/` directory. The first exon runs from the start of the sequence up to and including the sixty-third BASE, and the second exon runs from the ninety-first BASE to the end of the sequence.
2. "remote" DNA sequence: get the DNA sequence for accession AJ223353 from NCBI in plain fasta format, and remove the whole of the fasta header line.
   See the Bioinformatic Web lecture for a reminder on how to do this on the command line.
   You will need to use the annotation available with the sequence to determine which segments of the sequence encode the protein or are non-coding regions: we do NOT have intron/exon information for this sequence, only what is coding (CDS) or non-coding! **Hint:** the annotation ISN'T in the fasta format output!

For both of the above two sequences:

1. please do **NOT** use your Firefox/Safari/Chrome/Bing/Edge/whatever browser, or the file manager in MobaXterm, to get/move files around (please)! You **MUST** (mostly) know by now how to do this
2. write a Python programme/script that will split the genomic DNA into coding and non-coding parts, and write each of these sub-sequences to individual files AND two separate files (so that we also have one for coding sequences, and one for non-coding sequences)
3. your programme/script should make sure that all sequences are in upper case and are DNA
4. your fasta headers should include the length of the sequence
5. your output fasta file names should, where possible, be the same as the fasta header, followed by the ".fasta" suffix, but not include any spaces: this is Linux, remember!

6. your programme/script should include comments so that I/anyone can understand what you have done
7. your programmes/scripts should be committed in your personal git repository and pushed to GitHub

"Mission Two": running BLAST

1. using the files that you have generated in Mission One, run a BLAST analysis against the nematode database (nem) we made previously in the BLAST lecture.
2. from the outputs that you get, what was the best hit for each of the sequences input?

Give feedback
Ask for help

Give feedback
Ask for help

Give feedback
Ask for help

Possible answers

Doing it in a Linux shell

```
# Bits done with Linux command line
# Copy the "local" DNA sequence file to the right place
cd
mkdir Exercises/Lecture12
cd Exercises/Lecture12
```

```
# Copy the LOCAL DNA sequence file to the folder we have just made for this lecture
cp /localdisk/data/BPSM/Lecture12/plain_genomic_seq.txt .
wc plain_genomic_seq.txt
1 1 136 plain_genomic_seq.txt
cat plain_genomic_seq.txt
ATCGATCGATCGATCXtGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCG
```

oops, non-DNA bases seen .... some cleaning up will be required!

Getting the "remote" DNA sequence in fasta format for accession AJ223353 can be done in a number of different ways... see the Bioinformatic Web lecture for a reminder!

```
# "REMOTE" DNA sequence file in fasta format: Linux shell
# -qO means do it quietly and make the Output file AJ223353.fasta
wget -qO AJ223353.fasta "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=AJ223353&rettype=fasta&retmode=text"
```

```
# What did we get?
cat AJ223353.fasta
```

```
>gi|3255998|emb|AJ223353.1| Homo sapiens mRNA for histone H2B, clone pJG4-5-15
GGGGAGTGTGCTAACTATTAACGCTACGATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGC
TCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGGAAGAAGCGCAAGCGCAGCCGCAAGGAGAGCT
ATTCAGTGTATGTGTACAAGGTGCTGAAGCAGGTCCATCCCGACACCGGCATCTCTTCCAAGGCAATGGG
GATCATGAATTCCTTCGTCAACGACATCTTCGAGCGCATCGCAGGCGAGGCTTCCCGCCTGGCGCATTAC
AACAAGCGCTCGACCATCACCTCCAGGGAGATCCAGACGGCCGTGCGCCTGCTGCTTCCGGGGGAGCTGG
CCAAGCACGCCGTGTCGGAGGGCACCAAGGCCGTCACCAAGTACACCAGTTCCAAGTAACTTTGCCAAGG
GAGAGACATGAAGACAGAGGAGAAATGAATGCATAAAATAACTGATAATATGAATCTATACATAGAACTT
AGGAAGTCTCATCTGCCTGAAAATGACTGTGTGGATCCCACCCAAATCCAACTCATCCTGGTTTGCTGCA
CACTGGTTCATCAAAAGAAGGTTACCGAGGGGAAGGAACTAAAGGTGTTTGCACTTCATGTTACTTTTTG
AGTTTATAAACATAAAAACAGAATTTACTTCTGTTACAGACCTAGTTACTGGGAATTCATTACTTGCCAT
GGACTACCTTTGCTAAGAAAAGTCTGAATGAGAAGATGGCAGGACGTCTGAAAAAAAAAGTTATAATTAA
TAAAATCTGCGGAGAATTGTAAAAAAAAAAAAAAAAA
```

 # Remove fasta header line from the REMOTE file, create a new "no header" file

```
grep -v ">" AJ223353.fasta > AJ223353_noheader.fasta
```

 # Check that we have removed one line

```
wc AJ*.fasta
```

```
  14   21  900 AJ223353.fasta
  13   12  821 AJ223353_noheader.fasta
  27   33 1721 total
```

```
head -n1 AJ223353_noheader.fasta
```

```
GGGGAGTGTGCTAACTATTAACGCTACGATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGC
```

 # Simpler: get fasta, trim off header, save to file, all at the same time

```
wget -qO - "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=AJ223353&rettype=fasta&retmode=text" | grep -v ">" > AJ223353_noheader.fasta
```

Have you installed the `edirect` software package yet?!

It was the Bioinformatic Web lecture...

 # Use edirect tools to search for, then get, the sequence as fasta format,
 # then trim off header,
 # and save it to a file

```
esearch -db nucleotide -query "AJ223353[accession]" | efetch -format fasta | grep -v ">" > AJ223353_noheader.fasta2
```

```
wc -l AJ*
```

```
  14 AJ223353.fasta
  13 AJ223353_noheader.fasta
```

```
  12 AJ223353_noheader.fasta2
  39 total
```

"REMOTE" file in GenBank "standard" format

```
wget -qO AJ223353.genbank "https://eutils.ncbi.nlm.nih.gov/entrez/eutils/efetch.fcgi?db=nuccore&id=AJ223353&rettype=gb&retmode=text"
```

OR

```
esearch -db nucleotide -query "AJ223353[accession]" | efetch -format gb > AJ223353.genbank
```

cat AJ223353.genbank

```
LOCUS       AJ223353                 808 bp    mRNA    linear   PRI 07-
DEFINITION  Homo sapiens mRNA for histone H2B, clone pJG4-5-15.
ACCESSION   AJ223353
VERSION     AJ223353.1  GI:3255998
KEYWORDS    histone H2B.
SOURCE      Homo sapiens (human)
  ORGANISM  Homo sapiens
            Eukaryota; Metazoa; Chordata; Craniata; Vertebrata; Eutelec
            Mammalia; Eutheria; Euarchontoglires; Primates; Haplorrhini
            Catarrhini; Hominidae; Homo.
REFERENCE   1
  AUTHORS   Lorain,S., Quivy,J.P., Monier-Gavelle,F., Scamps,C., Leclus
            Almouzni,G. and Lipinski,M.
  TITLE     Core histones and HIRIP3, a novel histone-binding protein,
            interact with WD repeat protein HIRA
  JOURNAL   Mol. Cell. Biol. 18 (9), 5546-5556 (1998)
   PUBMED   9710638
REFERENCE   2  (bases 1 to 808)
  AUTHORS   Lipinski,M.
  TITLE     Direct Submission
  JOURNAL   Submitted (08-JAN-1998) Lipinski M., CNRS URA 1156, Institu
            Gustave Roussy, Rue Camille Desmoulins, 94805, FRANCE
FEATURES             Location/Qualifiers
     source          1..808
                     /organism="Homo sapiens"
                     /mol_type="mRNA"
                     /db_xref="taxon:9606"
                     /clone="pjG4-5-15"
                     /cell_line="HeLa"
                     /tissue_type="cervix carcinoma"
                     /dev_stage="adult"
     gene            1..808
                     /gene="HIRIP2"
     CDS             29..409
                     /gene="HIRIP2"
```

```
                     /codon_start=1
                     /product="Histone H2B"
                     /protein_id="CAA11277.1"
                     /db_xref="GI:3255999"
                     /db_xref="GOA:P58876"
                     /db_xref="HGNC:HGNC:4747"
                     /db_xref="InterPro:IPR000558"
                     /db_xref="InterPro:IPR007125"
                     /db_xref="InterPro:IPR009072"
                     /db_xref="UniProtKB/Swiss-Prot:P58876"
                     /translation="MPEPTKSAPAPKKGSKKAVTKAQKKDGKKRKRSRKE
                     VLKQVHPDTGISSKAMGIMNSFVNDIFERIAGEASRLAHYNKRSTITSRE
                     LPGELAKHAVSEGTKAVTKYTSSK"
     regulatory      769..774
                     /regulatory_class="polyA_signal_sequence"
                     /gene="HIRIP2"
ORIGIN
        1 ggggagtgtg ctaactatta acgctacgat gcctgaacct accaagtctg ctcctg
       61 aaagaagggc tccaagaagg cggtgactaa ggctcagaag aaggacggga agaagc
      121 gcgcagccgc aaggagagct attcagtgta tgtgtacaag gtgctgaagc aggtcc
      181 cgacaccggc atctcttcca aggcaatggg gatcatgaat tccttcgtca acgaca
      241 cgagcgcatc gcaggcgagg cttcccgcct ggcgcattac aacaagcgct cgacca
      301 ctccagggag atccagacgg ccgtgcgcct gctgcttccg ggggagctgg ccaagc
      361 cgtgtcggag ggcaccaagg ccgtcaccaa gtacaccagt tccaagtaac tttgcc
      421 gagagacatg aagacagagg agaaatgaat gcataaaata actgataata tgaatc
      481 catagaactt aggaagtctc atctgcctga aaatgactgt gtggatccca cccaaa
      541 actcatcctg gtttgctgca cactggttca tcaaaagaag gttaccgagg ggaagg
      601 aaaggtgttt gcacttcatg ttacttttg agtttataaa cataaaaaca gaattt
      661 ctgttacaga cctagttact gggaattcat tacttgccat ggactacctt tgctaa
      721 agtctgaatg agaagatggc aggacgtctg aaaaaaaaag ttataattaa taaaat
      781 ggagaattgt aaaaaaaaaa aaaaaaaa
//
```

 # We could just extract the CDS (coding segment) line using a pipe and grep

```
esearch -db nucleotide -query "AJ223353[accession]" | efetch -format gb | grep "CDS"
     CDS             29..409
```

Doing it all in Python

 # Start Python3

```
python3
```

 # Import modules; we can import all of them on the same line, comma separated

```
import os, subprocess, shutil
```

```
# Make ourselves a working directory for this lecture: what happens when we try this?
```

```
os.mkdir("/localdisk/home/aivens2/Exercises/Lecture12")
```

```
os.chdir("/localdisk/home/aivens2/Exercises/Lecture12")
```

```
os.listdir()
```

```
['MyFruityScript.py', 'MyNewDir', 'MyFruityScript_v2.py', 'my_dna.txt']
```

```
# Copy LOCAL DNA sequence file to the folder we have just made for this lecture
```

```
# Get the LOCAL DNA sequence file: method 1
```

```
subprocess.call("cp /localdisk/data/BPSM/Lecture12/plain_genomic_seq.txt   . ",shell=True)
```

```
0
```

```
# Get the LOCAL DNA sequence file: method 2
```

```
os.system("cp /localdisk/data/BPSM/Lecture12/plain_genomic_seq.txt   . ")
```

```
0
```

```
# Get the LOCAL DNA sequence file: method 3
```

```
shutil.copy("/localdisk/data/BPSM/Lecture12/plain_genomic_seq.txt" , "./shutilversion.txt" )
```

```
'./shutilversion.txt'
```

```
# What is in our directory now?
```

```
os.listdir()
```

```
['MyFruityScript.py', 'MyNewDir', 'MyFruityScript_v2.py', 'plain_genomic_seq.txt', 'shutilversion.txt', 'my_dna.txt']
```

```
print("\n".join(os.listdir()))
```

```
MyFruityScript.py
MyNewDir
MyFruityScript_v2.py
plain_genomic_seq.txt
shutilversion.txt
my_dna.txt
```

```
# What is in the file? \n
```

```
print(open("plain_genomic_seq.txt").read())
```

```
ATCGATCGATCGATCXtGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCGATCGATCGATCGATCGATCGA
```

```
# Show it after we have removed the last \n
print(open("plain_genomic_seq.txt").read().rstrip())
```

```
ATCGATCGATCGATCXtGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCGATCGATCGATCGATCGAT
```

```
# Store the local sequence in a variable: no print command this time
local_seq = open("plain_genomic_seq.txt").read().rstrip()
local_seq
```

```
'ATCGATCGATCGATCXtGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCGATCGATCGATCGATCGAT
```

```
# REMOTE file section
```

```
# Get the remote file and save locally
subprocess.call("esearch -db nucleotide -query \"AJ223353[accession]\" |
  efetch -format fasta | grep -v \">\" > AJ223353_noheader.fasta3",shell=True)
```

```
# Open and read the remote file, converting it to upper case as we do it
remotefile = open("AJ223353_noheader.fasta3").read().upper()
remotefile
```

```
'GGGGAGTGTGCTAACTATTAACGCTACGATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGC\nTCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGGAAGAAGCGCAAGCGCAGCCGCAAGGAGA
....
GAAAAGTCTGAATGAGAAGATGGCAGGACGTCTGAAAAAAAAAGTTATAATTAA\nTAAAATCTGCGGAGAATTGTAAAAAAAAAAAAAAAAAA\n'
```

```
len(remotefile)
```

```
820
```

```
# Open and read the local file, converting to upper case as we do it
local_seq = open("plain_genomic_seq.txt").read().upper()
local_seq
```

```
'ATCGATCGATCGATCXTGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCGATCGATCGATCGATCGATCG
```

```
len(local_seq)
```

```
136
```

```
len(local_seq.rstrip())
```

```
135
```

```
# What objects do we have in our python "bubble"
dir()
```

['__annotations__', '__builtins__', '__doc__', '__loader__', '__name__', '__package__', '__spec__', '__warningregistry__', 'local_seq', 'os', 'remotefile', 'shutil', 'subprocess']

```
# Convert remote file to single line
remotefile_singleline = remotefile.replace("\n","")
remotefile_singleline
```

```
'GGGGAGTGTGCTAACTATTAACGCTACGATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGCTCCAAGAAGGCG
```

```
# Sequence ready, make a "copy" for other analyses
remotefile_singleline_ready = remotefile_singleline
len(remotefile_singleline_ready)
```

```
808
```

```
# Convert local file to single line using "replace()
local_seq_singleline = local_seq.replace("\n","")
local_seq_singleline
```

```
'ATCGATCGATCGATCXTGSAKLGACTGACTAGTCATAGCTATGCXXXXXATGTAGCTACTCGATCGATCGATCGATCGATCG
```

```
len(local_seq_singleline)
```

```
135
```

Dealing with the "possibly not DNA" characters

We initially think that maybe they could be ambiguity codes for DNA, so we can check this quickly: IUPAC ambiguity codes.

IUPACCode	Meaning	Complement
A	A	T
C	C	G
G	G	C
T/U	T	A
M	A or C	K
R	A or G	Y

W	A or T	W
S	C or G	S
Y	C or T	R
K	G or T	M
V	A or C or G	B
H	A or C or T	D
D	A or G or T	H
B	C or G or T	V
N	G or A or T or C	N

A visible inspection of our sequence indicated that we have S, K, L and X, as well as GATC. Of these, **only S** is an official DNA ambiguity code, so we make a unilateral decision to remove everything that is not GATC from both of our sequences....as we are pretty sure they aren't protein!

```
# Look for, then remove, non-DNA characters from remote file
remotefile_singleline_anythingleft = remotefile_singleline.replace("G","").replace("A","").replace("T","").replace("C","
remotefile_singleline_anythingleft
''
```

```
# Look for, then remove, non-DNA characters from local file
local_seq_singleline_anythingleft = local_seq_singleline.replace("G","").replace("A","").replace("T","").replace("C","")
local_seq_singleline_anythingleft
XSKLXXXXX
local_seq_singleline_reallyDNA = local_seq_singleline.replace("X","").replace("S","").replace("K","").replace("L","")
local_seq_singleline_reallyDNA
'ATCGATCGATCGATCTGAGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGATCGATCGATCGATCGA
```

YUK! There **has** to be a better way?! (yes there is, for example using things like:

```
set(list(local_seq.rstrip()))
```

```
{'A', 'L', 'S', 'K', 't', 'C', 'G', 'T', 'X'}
```

or the `re` module, but we haven't done these yet... soon, patience..!)

```
len(local_seq_singleline_reallyDNA)
```

```
126
```

```
len(local_seq_singleline)
```

```
135
```

```
# Sequence ready for other analyses: essentially copying the variable
local_seq_singleline_ready = local_seq_singleline_reallyDNA
```

```
# REMOTE sequence coding/noncoding
# GenBank format output says CDS is BASES 29..409, so the rest is not protein coding
remote_noncoding01 = remotefile_singleline_ready[0:28]
remote_exon01 = remotefile_singleline_ready[28:409]
remote_noncoding02 = remotefile_singleline_ready[409:]
```

```
# What are the remote sub-sequences?
remote_noncoding01
```

```
'GGGGAGTGTGCTAACTATTAACGTACG'
```

```
remote_exon01
```

```
'ATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGCTCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGG
```

```
remote_noncoding02
```

```
'CTTTGCCAAGGGAGAGACATGAAGACAGAGGAGAAATGAATGCATAAAATAACTGATAATATGAATCTATACATAGAACTTA
```

```
# Quick check to see that we have everything in our subsets
len(remote_noncoding01) + len(remote_exon01) + len(remote_noncoding02)
```

```
808
```

```
# LOCAL sequence coding/noncoding
# We are told that the first exon runs from the start of the sequence upto and including the sixty-third BASE
# The second exon runs from the ninety-first BASE to the end of the sequence.
local_exon01 = local_seq_singleline_ready[0:63]
local_intron01 = local_seq_singleline_ready[63:90]
local_exon02 = local_seq_singleline_ready[90:]
```

```
# What are the local sub-sequences?
local_exon01
'ATCGATCGATCGATCTGAGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGA
local_intron01
'CGATCGATCGATCGATCGATCGATCAT'
local_exon02
'GCTATCATCGATCGATATCGATGCATCGACTACTAT'
```

```
# Quick check to see that we have everything in our subsets
len(local_exon01) + len(local_intron01) + len(local_exon02)
126
```

```
# Write out the various files with something
# sensible in their headers, plus lengths as requested
# We could write out to the "combined" files at the same time,
# but as this is our first go, we'll do things sequentially
```

```
# REMOTE ones first
remote_noncoding01_out = open("remote_noncoding01.fasta", "w")
remote_noncoding01_out.write(">AJ223353_noncoding01_length" + str(len(remote_noncoding01)) + "\n")
remote_noncoding01_out.write(remote_noncoding01)
remote_noncoding01_out.close()
print(open("remote_noncoding01.fasta").read())
```

```
>AJ223353_noncoding01_length28
GGGGAGTGTGCTAACTATTAACGCTACG
```

```
# remote_exon01
remote_exon01_out = open("remote_exon01.fasta", "w")
remote_exon01_out.write(">AJ223353_exon01_length" +
str(len(remote_exon01)) + "\n")
remote_exon01_out.write(remote_exon01)
remote_exon01_out.close()
print(open("remote_exon01.fasta").read())
>AJ223353_exon01_length381
ATG CCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGCTCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGGA
```

```
# remote_noncoding02
remote_noncoding02_out = open("remote_noncoding02.fasta", "w")
remote_noncoding02_out.write(">AJ223353_noncoding02_length" +
str(len(remote_noncoding02)) + "\n")
remote_noncoding02_out.write(remote_noncoding02)
remote_noncoding02_out.close()
print(open("remote_noncoding02.fasta").read())
>AJ223353_noncoding02_length399
CTTTGCCAAGGGAGAGACATGAAGACAGAGGAGAAATGAATGCATAAAATAACTGATAATATGAATCTATACATAGAACTTAC
```

```
# LOCAL ones next
local_exon01_out = open("local_exon01.fasta", "w")
local_exon01_out.write(">LocalSeq_exon01_length" + str(len(local_exon01))
+ "\n")
local_exon01_out.write(local_exon01)
local_exon01_out.close()
print(open("local_exon01.fasta").read())
>LocalSeq_exon01_length63
ATCGATCGATCGATCTGAGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGA
```

```
# Now intron 1
local_intron01_out = open("local_intron01.fasta", "w")
local_intron01_out.write(">LocalSeq_intron01_length" +
str(len(local_intron01)) + "\n")
```

```
local_intron01_out.write(local_intron01)
local_intron01_out.close()
print(open("local_intron01.fasta").read())
>LocalSeq_intron01_length27
CGATCGATCGATCGATCGATCGATCATGCT
```

```
# Now exon 2
local_exon02_out = open("local_exon02.fasta", "w")
local_exon02_out.write(">LocalSeq_exon02_length" + str(len(local_exon02)) + "\n")
local_exon02_out.write(local_exon02)
local_exon02_out.close()
print(open("local_exon02.fasta").read())
>LocalSeq_exon02_length36
GCTATCATCGATCGATATCGATGCATCGACTACTAT
```

```
# COMBINED FASTA files
# Now make the "coding regions" file
# We could just read back the files, but as this is our first go
# we will just add the bits sequentially,
# putting extra EOL (End Of Line) characters in
exons_out = open("All_exons.fasta", "w")
exons_out.write(">AJ223353_exon01_length" + str(len(remote_exon01)) + "\n" + remote_exon01 + "\n")
exons_out.write(">LocalSeq_exon01_length" + str(len(local_exon01)) + "\n" + local_exon01 + "\n")
exons_out.write(">LocalSeq_exon02_length" + str(len(local_exon02)) + "\n" + local_exon02)
exons_out.close()
print(open("All_exons.fasta").read())
>AJ223353_exon01_length381
ATGCCTGAACCTACCAAGTCTGCTCCTGCCCCAAAGAAGGGCTCCAAGAAGGCGGTGACTAAGGCTCAGAAGAAGGACGGGAA
>LocalSeq_exon01_length63
ATCGATCGATCGATCTGAGACTGACTAGTCATAGCTATGCATGTAGCTACTCGATCGATCGAT
>LocalSeq_exon02_length36
GCTATCATCGATCGATATCGATGCATCGACTACTAT
# Now make the "non-coding regions" file
```

```
introns_out = open("All_noncodings.fasta", "w")
introns_out.write(">AJ223353_noncoding01_length" +
str(len(remote_noncoding01)) + "\n" + remote_noncoding01 + "\n")
introns_out.write(">AJ223353_noncoding02_length" +
str(len(remote_noncoding02)) + "\n" + remote_noncoding02 + "\n")
introns_out.write(">LocalSeq_intron01_length" + str(len(local_intron01)) +
"\n" + local_intron01)
introns_out.close()
print(open("All_noncodings.fasta").read())
```

```
>AJ223353_noncoding01_length28
GGGGAGTGTGCTAACTATTAACGCTACG
>AJ223353_noncoding02_length399
CTTTGCCAAGGGAGAGACATGAAGACAGAGGAGAAATGAATGCATAAAATAACTGATAATATGAATCTATACATAGAACTTAGGAAGTCT
>LocalSeq_intron01_length27
CGATCGATCGATCGATCGATCGATCAT
```

```
# Does Python an equivalent to history in bash?
# Yes, we can write a small function (later lecture...)
def history(howmany=20) :
  import readline
  his_start = readline.get_current_history_length()-howmany
  his_stop = readline.get_current_history_length()
  for this in range(his_start,his_stop) :
    print("["+str(this)+"]",readline.get_history_item((this + 1)))
```

```
# Now just "call" the function....last 8 commands...
history(8)
```

```
[3800] print(open("All_noncodings.fasta").read())
[3801] def history(howmany=20) :
[3802]   import readline
[3803]   his_start = readline.get_current_history_length()-howmany
[3804]   his_stop = readline.get_current_history_length()
[3805]   for this in range(his_start,his_stop) :
[3806]     print("["+str(this)+"]",readline.get_history_item((this + 1)))
[3807] history(8)
```

```
Finished in Python, could use CTRL-D or exit()
exit()
```

```
# Back in Linux now!
# What fasta files did we create?
ls -1 *.fasta*
```

```
AJ223353.fasta
AJ223353_noheader.fasta
AJ223353_noheader.fasta2
AJ223353_noheader.fasta3
All_exons.fasta
All_noncodings.fasta
local_exon01.fasta
local_exon02.fasta
local_intron01.fasta
remote_exon01.fasta
remote_noncoding01.fasta
remote_noncoding02.fasta
```

```
# What fasta headers do we have in the exons file?
```

```
grep ">" All_exons.fasta
```

```
>AJ223353_exon01_length381
>LocalSeq_exon01_length63
>LocalSeq_exon02_length36
```

```
# What fasta headers do we have in the introns file?
```

```
grep ">" All_noncodings.fasta
```

```
>AJ223353_noncoding01_length28
>AJ223353_noncoding02_length399
>LocalSeq_intron01_length27
```

```
# Run blastx (DNA query vs a protein database) against our nem database
# which should still be in your ~/Exercises/Lecture06 directory
```

```
# Syntax for the "nice tabular format" is:
```

```
blastx -db databasename -query fastafilename -outfmt 7
```

```
# We can parse the blast output "on the fly" as we only want the best "hit".
# We have 5 commented out lines at the top of the BLAST output, so we'd want up to line 6, as that is the first "hit"
```

```
blastx -db databasename -query fastafilename -outfmt 7 | head -n6
```

```
# The lines we might actually run
```

```
db="/localdisk/home/aivens2/Exercises/Lecture06/nem" # variable for the database
```

```
queryseqs="All_exons.fasta" # variable for the file of all exons
```

```
blastx -db ${db} -query ${queryseqs} -outfmt 7 | head -n6
```

```
# BLASTX 2.17.0+
# Query: AJ223353_exon01_length381
# Database: /localdisk/home/aivens2/Exercises/Lecture06/nem
# Fields: query acc.ver, subject acc.ver, % identity, alignment length, mismatches, gap op
# 38 hits found
AJ223353_exon01_length381       gi|341891843|gb|EGT47778.1|     92.308  91      7       0
```

That wasn't what we wanted, as there were three exon sequences, but only

results for one!

Let's modify some parameters to change the output from BLAST
`-outfmt 6` (this gives us the tabulated format, but no heading information)
`-num_alignments 1` (this will only give us the single best alignment for each of our three exon sequences)

```
blastx -db ${db} -query ${queryseqs} -outfmt 6 -num_alignments 1 > quick_blast_check.out
```

```
Warning: [blastx] Examining 5 or more matches is recommended
```

```
cat quick_blast_check.out
```

```
AJ223353_exon01_length381       gi|341891843|gb|EGT47778.1|     92.308  91      7       0       106
LocalSeq_exon02_length36        gi|373218841|emb|CCD63632.1|    66.667  9       3       0       10
```

Why only two results?!

And so on for the rest of the individual fasta files: could maybe use a

while read ...
do
...
done < list_of_files

loop in a bash script to process the BLAST output from the multiple queries (i.e. when we have many to run and process)?

Or better still, if only we knew how to use a loop within Python, it would make things SOOOOOOO much easier... so let's learn how to use them: the next lecture!

git and GitHub time!

```
# You might want to add other files?
```

```
git st
```

```
On branch master
Changes not staged for commit:
  (use "git add/rm <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        deleted:    ../Lecture04/CodeFiles.tar.gz
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
```

```
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
        ../Lecture07/myfileofaccessions.txt
        ./

no changes added to commit (use "git add" and/or "git commit -a")
```

ls

```
AJ223353.fasta             AJ223353_noheader.fasta3  local_exon02.fasta    MyFruityScript_v2.py   re
AJ223353.genbank           All_exons.fasta           local_intron01.fasta  MyNewDir               re
AJ223353_noheader.fasta    All_noncodings.fasta      my_dna.txt            plain_genomic_seq.txt  re
AJ223353_noheader.fasta2   local_exon01.fasta        MyFruityScript.py     quick_blast_check.out  sh
```

git add *.py

git commit -m "Scripts from the second BPSM python lecture"

```
[master e444ef8] Scripts from the second BPSM python lecture
 2 files changed, 26 insertions(+)
 create mode 100755 Lecture12/MyFruityScript.py
 create mode 100755 Lecture12/MyFruityScript_v2.py
```

Push our local repository to the one created on GitHub

git push AllLectures master

```
Enumerating objects: 6, done.
Counting objects: 100% (6/6), done.
Delta compression using up to 256 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 787 bytes | 393.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0
remote: Resolving deltas: 100% (2/2), completed with 1 local object.
To https://github.com/B123456-2121/BPSM_LectureStuff.git
   6d6f5e7..e444ef8  master -> master
```

git st

```
On branch master
Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
        modified:   ../Lecture06/run_this_script_v1

Untracked files:
  (use "git add <file>..." to include in what will be committed)
        ../BN.out
        ../BN.sh
        ../Lecture04/Als_ICA/
        ../Lecture04/all_the_things_I_did
        ../Lecture04/outs/
        ../Lecture05/Country.Mozambique.details
        ../Lecture07/Cosmoscarta.nuc.fa
        ../Lecture07/Cosmoscarta.nuc.gis
        ../Lecture07/myfileofaccessions.txt
        AJ223353.fasta
        AJ223353.genbank
        AJ223353_noheader.fasta
```

```
        AJ223353_noheader.fasta2
        AJ223353_noheader.fasta3
        All_exons.fasta
        All_noncodings.fasta
        local_exon01.fasta
        local_exon02.fasta
        local_intron01.fasta
        my_dna.txt
        plain_genomic_seq.txt
        quick_blast_check.out
        remote_exon01.fasta
        remote_noncoding01.fasta
        remote_noncoding02.fasta
        shutilversion.txt

no changes added to commit (use "git add" and/or "git commit -a")
```

git lo

```
e444ef8 (HEAD -> master, AllLectures/master) Scripts from the second BPSM python lecture
6d6f5e7 My python scripts from the first BPSM python lecture
eae65c6 (Lecture07/master) edirect seqs related stuff added
eea4f5e (Lec_6_branch) Kept bash script output files from Lecture 06
fbe5cc0 Added run_this_script_v1 from BPSM Lecture06
ce2c15a nematode BLASTDB stuff added
19cdd2d Added the example_people_data.tsv file from Lecture03
137c3db Decided to keep blastoutput2.out
08d379b Decided to keep myinputfile.tsv
c9b5b18 First attempt at parsing BLAST data
bbf825c My first awk script, Lecture05
68f27fb There were conflicts with someotherfile.sh, kept all changes
83629cb (Version1.2) Final version of the someotherfile.sh shell script prior to merge
0b5425b (tag: v2.0) Updated contents of tar, coolscript
5d8aeea I am not sure cool script is actually any good
bbd36e7 Added my_cool_script
ea60357 Third version of the someotherfile.sh shell script
b6ae4a7 Updated version of tar
ce230e1 (tag: v1.2) Second version of the someotherfile.sh shell script
a5add7c Added the someotherfile.sh shell script
101b700 Additional motif files, and updated tar file
d21cb3d Adding intial tar file
59d1756 CodeFiles all done and tar zipped
6dd85a8 First file added
```

Have your say...

The "Postbox of Truth": is your chance to give anonymous "in course" feedback/comments. Your comments are completely anonymous and confidential, so say what you think, irrespective of whether the comments are positive or otherwise! You can use this multiple times too: I use it throughout the course to enable you to give context-relevant feedback/make suggestions.

Just click the image!

Sources used